

Part 3 — Proof of concept, blockers, schedule and release

Summary

The proof of concept runs the Part 1 crawler through the Part 2 pipeline in one region and answers the questions the design rests on. It is two runs. **Run A** fetches up to a million URLs on the three hosts the brief names, twice, a week apart, to measure what those hosts tolerate, how often they block, and how often pages change. **Run B** fetches 25 million URLs across at least ten thousand hosts to measure fleet throughput, unit costs and database behaviour, with synthetic loads for the parts a 25-million run cannot reach.

Ten calendar weeks, two backend engineers, 20 engineer-weeks as the likely case (14 low, 30 high), in five gated phases: measure, build the pipeline, run at scale, add the read path, harden. The two things that decide what can be promised are measured in week one and settled by the customer, not engineering: how fast the target hosts will let us fetch, and whether disguised blocks are to be accepted as a coverage ceiling.

Every pass mark is a number with a method. The PoC passes when every row in §2 has a measured value, each pass/fail row passes, and the Part 2 cost estimate re-run with the measured inputs lands within the stated $\pm 50\%$, or the difference is explained line by line.

The brief's three outputs map as follows: how to proceed to next steps → §4 and §7; how to evaluate the proof of concept → §1 and §2; release plan and time estimations → §5 and §6.

1. What the proof of concept covers

Run	Input	Measures	Why this shape
A — few hosts	the three hosts the brief names; up to 1M URLs from their sitemaps or a customer list; two cycles a week apart	accepted request rate per host; block rate and its disguises; page-size distribution; how many pages are unchanged a week later; the share needing JavaScript	the brief's own example; at 10 requests a second per host, 1M URLs take about nine hours, so this run cannot test fleet throughput and is not asked to
B — many hosts	25M URLs across $\geq 10,000$ hosts, from the customer's long tail or hosts whose robots.txt permits crawling; one cycle	sustained fetch rate, parse cost on the real server, unit costs, ClickHouse insert behaviour, lookup latency	throughput can only be measured where politeness is not the ceiling; 25M URLs is about seven hours at 1,000 a second
Synthetic loads	a frontier table filled with a billion generated rows over a million hosts; an inserter writing 3,900 rows a second into ClickHouse	frontier refill latency; insert lag and part counts at the production rate	a 25M-URL run cannot reach either number

Out of scope: a second region, JavaScript rendering, the recrawl scheduler beyond the priority rule, and any read API beyond single-URL lookup and the host-and-month report.

2. How the proof of concept is judged

Every metric has a method and a pass rule. "Measured, no target" is deliberate where a target would create the wrong pressure.

Extraction and classification. A labelled sample of 500 pages across the three hosts and the long tail, labelled by one engineer who did not write the extractor and one subject-matter reviewer from the customer, with disagreements adjudicated and agreement reported before the labels are used. 500 gives a 95% target an accuracy of about ± 2 points at 95% confidence; the labelling is about one engineer-week and is budgeted in phase 2.

Metric	Pass
Title correct	$\geq 98\%$
Page type correct	$\geq 95\%$, confusion matrix reported
Body free of navigation, ads and footer	$\geq 90\%$
Topics: at least three of the top five judged relevant by both reviewers	$\geq 70\%$
Invented metadata (an author or date not present in the page)	none

Throughput, cost and the design's own objectives (Run B and the synthetic loads).

Metric	Pass
Sustained fetch rate	$\geq 1,000$ URLs a second for six hours on at most 70 servers
Parse cost per page on the real server	measured; replaces the 0.75 s assumption
Released URL fetched within one hour; fetch visible to queries within ten minutes	$\geq 99\%$; $\geq 99\%$
Parse latency	95th percentile under 1.5 s
Rate holding	each host's fetch rate within 5% of its cap over an hour; no 429s at the cap
Unit costs per line (server-hours, object writes, bytes stored)	measured and written back into the cost model; no percentage gate at this scale, where Spot price noise exceeds it
ClickHouse parts per partition; insert lag	under 300 parts; lag under five minutes at 3,900 rows a second
Single-URL lookup latency with inserts running	99th percentile under 100 ms, or the key-value summary tier is added
Frontier refill latency at a million hosts	sustains the release rate, or queue heads are pre-built by a batch job

Coverage (Run A).

Metric	Pass
Accepted rate per host, probed at 1, 5 and 10 requests a second	measured, no target: the negotiation starts from it
Block rate per host, including <code>no_content</code> hosts where a browser-User-Agent probe succeeds	measured, no target: a target would push toward evading detection
Error rate per host, excluding blocks	under 0.5%
Unchanged rate on the second cycle, per host	measured; replaces the 60% assumption behind the largest saving
Share of pages needing JavaScript, per host	measured; above 10% triggers a separately funded rendering workstream
Page-size distribution	measured; replaces the 200 KB assumption

Operations (phase 4).

Test	Pass
Kill 20% of workers mid-run	no URL lost, no duplicate row
Fail over Redis during a run	at most an hour of URLs re-released, none lost

Test	Pass
Pause consumers for two hours, resume	backlog drained within two hours
One-week alert soak	every alert actionable
Each runbook	completed by someone who did not write it

3. Blockers

Class: **known** means the method is settled and only effort remains; **risky** means the outcome is uncertain and a measurement or a decision resolves it. Arrival is when it is expected to be resolved on the §4 schedule.

Decided outside engineering. These gate the plan and have to be raised on day one.

#	Blocker	Class	Arrival	Resolved by
1	Legal position on crawling hosts that disallow it, and on probing with a browser User-Agent	risky	entry gate for phase 0; owner named before day one	a written position from counsel before any probe
2	Anti-bot blocking on the target hosts, including blocks disguised as maintenance pages	risky	measured week 1; decision week 2	the customer decides: accept the coverage ceiling, or negotiate access
3	The request rate each host will accept	risky	measured week 1; negotiation open-ended	anything above the probed rate is a commercial agreement
4	Spot capacity quota (Run B needs about 800 cores; production about 4,000)	known, external	filed day 1; days to weeks	quota request; run smaller until granted
5	Customer MySQL access: replica, network path, credentials	known, external	week 1	customer provisions; ingest from file until then
6	PoC infrastructure budget (about \$15–20k over ten weeks at list prices)	known, external	before phase 1	approval by the budget owner named at the phase-0 gate
7	Security review of a fleet with public egress addresses	known, external	before phase 2	request filed in week 1

Measured by the PoC. Each replaces an assumption in Part 2.

#	Blocker	Class	Arrival	Resolved by
8	Page-size distribution	risky	week 1	Run A sample of 10,000 pages; a 5× swing moves storage and bandwidth
9	Parse cost on the real server	risky	week 1	profiling in the fleet; sets the largest cost line
10	Unchanged rate on recrawl	risky	week 6	Run A second cycle; sets the largest saving
11	Share of pages needing JavaScript	risky	week 1	detector on the Run A sample
12	Egress address reputation	risky	week 4	block rate by egress address in Run B; reverses to fixed addresses if a host needs an allow-list
13	Extraction and topic quality	risky	week 6	the labelled sample in §2
14	ClickHouse insert behaviour at the production rate	risky	week 5	synthetic inserter; decides whether ClickHouse or a managed alternative ships
15	Single-URL lookup latency under inserts	risky	week 7	load test; decides whether the summary tier is added

#	Blocker	Class	Arrival	Resolved by
16	Frontier refill latency at a million hosts	risky	week 5	synthetic frontier; falls back to pre-built queue heads

Built by the PoC. Known work; the estimates are assumptions until phase 1 ends.

#	Blocker	Arrival	Effort
17	Ingesting an 800 GB list: parallel read, sort, dedup, load	week 2	3 days
18	Reading the list from MySQL	week 2	1 day
19	Malformed input: bad encodings, null bytes, stray line endings; reject and count, never drop silently	week 2	1 day
20	Frontier: table, refill, per-host queues, rate limiters, release	week 3	5 days
21	Raw-object packing with offset pointers	week 3	2 days
22	Fleet deployment and autoscaling	week 3	4 days
23	Monitoring and dashboards	week 5	4 days

4. Schedule

Ten weeks, five phases. Each phase de-risks something named and ends at a gate that must pass before the next begins. Phase 2 has three weeks because Run A needs two cycles a week apart and Run B follows them; phase 4 has three because failure injection produces findings that need fixing.

Phase	Weeks	Delivers	De-risks
0 — Measure	1	Run A sample of 10,000 pages: page sizes, block rate, JavaScript share, rate probes; parse cost on the real server; cost estimate re-run	blockers 2, 3, 8, 9, 11
1 — Pipeline	2–3	ingest from file and MySQL, frontier and queues, transport, worker fleet, object packing, ClickHouse schema; one region, fixed fleet size	blockers 17–22; that the parts fit together
2 — Scale runs	4–6	Run A in full, two cycles; Run B; the synthetic loads; insert tuning; monitoring wired	blockers 10, 12, 14, 16, 23
3 — Serving	7	read API, cache, CDN; lookup and report queries; load test	blocker 15
4 — Hardening	8–10	autoscaling, failure injection, runbooks, alert review, canary path, on-call	operability by people who did not build it

Exit gates:

1. **Phase 0.** Page size, parse cost and accepted rate measured per host; cost estimate revised and re-approved if any line moved more than 50%; the anti-bot question put to its owner.
2. **Phase 1.** 10,000 URLs end to end; for each URL the stored row and a Part 1 command-line run within the same hour agree on status, reason, robots state, HTTP status, title and page type (body and word count compared only on hosts that do not vary per fetch, which Amazon does); a killed worker loses no URL.
3. **Phase 2.** Every §2 throughput, cost and coverage row has a measured value; the revised cost estimate is within the stated range or explained.
4. **Phase 3.** Cached reads under 200 ms and misses under one second at the 95th percentile, at 250 requests a second for an hour with inserts running; the 99th percentile on misses recorded against the 100 ms threshold.
5. **Phase 4.** Every §2 operations row passes; runbooks executed by a non-author; rollback rehearsed.

5. Estimates

Two backend engineers, full time, with the cloud account provisioned and a customer contact who can answer the blocking and rate questions in week one. No JavaScript rendering, no second region, no customer-facing interface.

Phase	Low	Likely	High	What moves it
0 — Measure	1.5	2	2.5	probes repeated if a host blocks early
1 — Pipeline	3	4	5	frontier correctness: refill, release, re-release after a Redis loss
2 — Scale runs	4	6	9	two Run A cycles, Run B, synthetic loads, insert tuning, the labelled sample
3 — Serving	1.5	2	3.5	lookup latency forcing the summary tier, about a week
4 — Hardening	4	6	10	failure-injection findings; a replay run for the failure tests
Total, engineer-weeks	14	20	30	

Twenty engineer-weeks over ten calendar weeks is the two engineers at full allocation; there is no hidden slack, and the high case is fifteen calendar weeks. A third engineer with ClickHouse experience from week 3 brings the likely case back to eight. The estimates cover engineering only; the rate negotiation and the legal position run in parallel and can gate scale-up regardless.

Three assumptions would move the whole table: that the Part 1 extractor's quality is acceptable on the target hosts as it is; that the customer's list arrives as a file or a MySQL table without a new connector; and that no host requires authenticated or rendered access for its core pages.

6. Release

Staged rollout. Each stage runs beside the previous fleet; the scheduler weights each host's releases between the two, so rolling back is setting the new fleet's weight to zero, not redeploying.

Stage	Share	Duration	Proceed when
Canary	at most 10% of every host's budget on the new fleet	48 hours	error rate under 1%, parse failures under 5%, block rate per host within PoC bounds, no paging alert
Limited	10% of the batch, all hosts	1 week	unit costs within 20% of the estimate; objectives met
Broad	50%	1 week	objectives met; no manual intervention
Full	100%		error budget on plan

Automatic rollback when the error rate exceeds 1% for fifteen minutes on a first release (twice the seven-day baseline once one exists); parse failures exceed 5%; rows missing required fields exceed 0.1%; or an hourly cost proxy (server-hours × Spot price per URL fetched) exceeds 150% of the estimate for six hours. Cost Explorer lags a day and cannot drive a trigger.

Quality gates on every change: lint, strict types and tests green on every commit; the labelled sample re-run on every extractor or classifier change, with a drop of more than one point blocking the merge; the cost model re-run on every change to a capacity or price input; a canary before any fleet-wide deploy.

Runbooks, written before launch and each executed once by a non-author in phase 4: queue backlog growth; a host starting to block; ClickHouse insert lag; parse-failure spike after a site redesign; Spot capacity shortfall; Redis failover; rollback.

On-call. This is a batch system with a day of completion slack, so paging is limited to read-API availability and data-loss classes; everything else is a business-hours ticket. Two engineers plus one trained in phase 4 is a three-person rota, which is thin: each person is on call two weeks in three until the team grows, and the plan says so rather than pretending otherwise.

What high quality means here. Correct over complete: a missing field is acceptable, an invented one is not. Honest coverage: every batch report states fetched, blocked by reason, errored by reason, disallowed and JavaScript-only counts per host. Operable by others: runbooks, dashboards and alerts tested by people who did not build the system before it takes real load.

7. Next steps after the proof of concept

In the order that would change the design most:

1. Settle the accepted rate and the anti-bot policy with the customer; everything promised for the few-host case follows from these.
2. Re-run the cost estimate with the measured page size, parse cost and unchanged rate, and re-approve the budget.
3. Decide the JavaScript-rendering workstream from the measured share.
4. Decide the key-value summary tier from the measured lookup latency.
5. Run the fair column-store versus row-store comparison on real hardware before quoting any speed multiple.
6. Add a second region and a cross-region read replica only after the single-region objectives have held for a full monthly batch.